

Fundamentos de Criptografia e Segurança da Informação

Licenciatura em Ciências da Computação

José Bacelar Almeida

Criptografia

Preâmbulo

Conceitos Básicos

Criptografia Aplicada

Fundamentos de Criptografia e Segurança da Informação

Apresentação da tecnologia criptográfica como instrumento para a segurança da informação.

- O que é que a tecnologia criptográfica moderna nos pode oferecer no âmbito da segurança da informação?
- Quais são as premissas sobre as quais essas garantias de segurança são estabelecidas?
- Argumentação formal da segurança de primitivas criptográficas (modelos e provas de segurança);
- Ênfase nas Primitivas Assimétricas e **Criptografia Pós-Quântica**;
- Aplicações.

Conteúdos programáticos

1. Conceitos Preliminares e Terminologia
2. Criptografia Simétrica
3. Criptografia Assimétrica
4. Modelos de Segurança
5. Criptografia Pós-Quântica
6. Aplicações Correntes da Criptografia

Avaliação

- Componente Laboratorial (40%)
 - Guiões a realizar por grupos de 3 elementos nas práticas laboratoriais
 - Recurso ao GitHub para suporte e registo das actividades desenvolvidas.
 - Avaliação individual que incide sobre:
 - Trabalho realizado no período lectivo (pressupõe, portanto, controle de assiduidade);
 - Resposta a pequenos desafios lançados durante o semestre.
 - Projecto individual **opcional**.
 - Avaliação: $\max(\min(AC, 16), 0.8*AC + 0.2*PI)$
AC = Avaliação Contínua
PI = Projecto Individual
- Teste Escrito (60%, nota mínima 8/20)
- Em qualquer das componentes, os alunos podem ser chamados a uma prova oral, sendo, nesse caso, a nota final estabelecida com base nessa prova.

Horário / Calendário

- Componente Laboratorial:
 - PL1 - 2^a-feira, 9:00-11:00, Ed. 1 - 0.18
 - PL2 - 2^a-feira, 11:00-13:00, Ed. 1 - 2.05
- Aulas Teóricas:
 - 3^a-feira, 14:00-16:00, Ed. 1 - 0.22
- Teste final: 19 Maio (manhã)
- Exame de recurso; etc. : ???

Criptografia

Preâmbulo

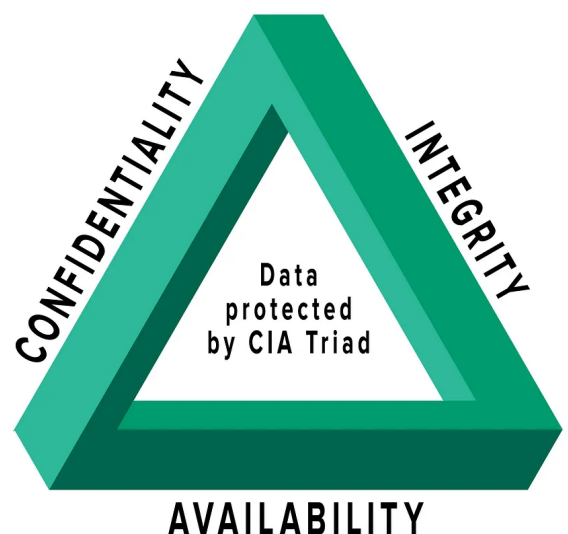
Conceitos Básicos

Criptografia Aplicada

Segurança da Informação

CIA triad

- **Confidencialidade:** assegurar as restrições impostas ao acesso e à divulgação de informações, incluindo meios para proteger a privacidade pessoal e as informações proprietárias. Uma perda de confidencialidade é a divulgação não autorizada de informações.
- **Integridade:** proteger contra modificação ou destruição inadequadas de informações, incluindo garantir o não repúdio e a autenticidade da informação. Uma perda de integridade é a modificação ou destruição não autorizada de informações.
- **Disponibilidade:** garantir acesso e uso oportunos e confiáveis da informação. Uma perda de disponibilidade é a interrupção do acesso ao uso de dados de um sistema de informação.



Criptografia & Segurança da Informação

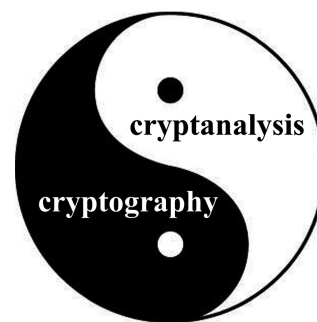
- “A criptografia é uma importante ferramenta de segurança de computadores que lida com técnicas para armazenar e transmitir informações de maneiras que impeçam acesso ou interferência não autorizados” [ISO]
- A criptografia serve como uma linha crucial de defesa contra várias ameaças, incluindo acesso não autorizado, violações de dados, adulteração e espionagem.
- A criptografia é um mecanismo de segurança que, usado em combinação com outros, permite atingir os objetivos de segurança da informação.

Origens do conceito

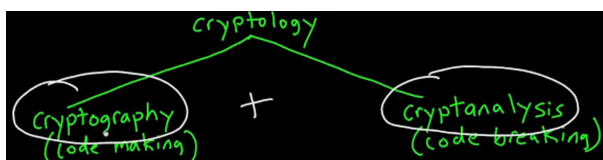
- "Kryptós" + "Gráphein" (Escrita secreta)
- ...arte de transmitir "segredos" (num canal aberto)
- Usado desde a antiguidade, frequentemente associado a atividades militares e diplomáticas.
- Hoje em dia, os objetivos alargaram-se para abranger aspectos adicionais do processo comunicativo num ambiente hostil:
 - **Confidencialidade:** sigilo de mensagens transmitidas;
 - **Autenticidade:** Estabelecer a origem das mensagens recebidas;
 - **Integridade:** a mensagem transmitida não foi adulterada;
 - ...
- Obs.: note que os âmbitos das propriedades foram reduzidos em relação ao seu uso no contexto da Segurança da Informação.



Cripto-análise



- Em contrapartida, a **Cripto-análise** visa minar os objetivos da criptografia.
- Uma técnica criptográfica é considerada **quebrada** se seus objetivos tiverem sido comprometidos (por meio de cripto-análise).
- Criptografia e Cripto-análise formam uma área por vezes designada **Criptologia**.
- Como disciplina científica, a criptografia cobre várias áreas diferentes, como teoria das probabilidades, teoria da complexidade, teoria da informação, ciência da computação, etc..



Breve história

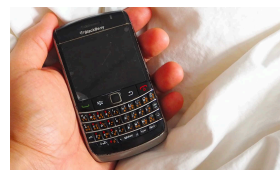
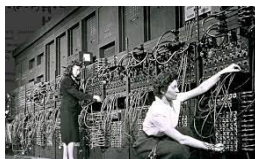


- O uso de (proto)cifras é conhecido desde tempos imemoriais (século XV AC).
 - E.g., exércitos de impérios clássicos (e.g., romanos) usavam regularmente criptografia
- A sofisticação das técnicas evoluiu ao longo dos séculos
 - E.g.: máquina Enigma - WW2, Alemanha.
- Logo após a Segunda Guerra Mundial, *Claude Shannon* ([vídeo](#)) publicou dois artigos seminais sobre **Teoria da Informação**, que permitiram a formalização da segurança de cifras — evento considerado o nascimento da Criptografia como uma área científica.

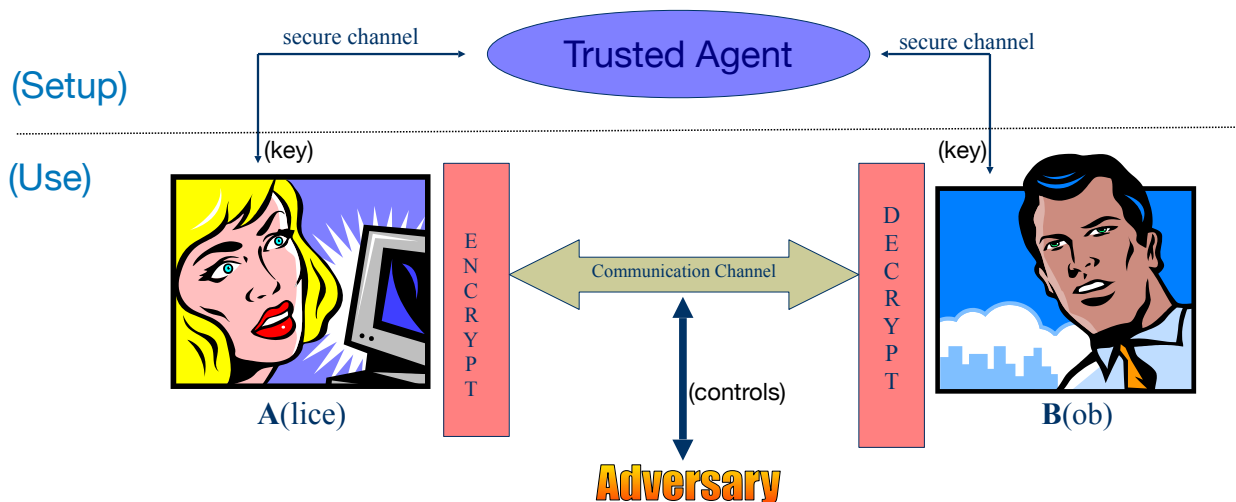


Modern Cryptography

- 1948/9 – Teoria da Informação (Claude Shannon).
- 1970/7 – Data Encryption Standard (DES).
- 1976 – Criptografia de Chave-Pública (Diffie & Hellmann)
- 2001 – Escolha do substituto do DES: Advanced Encryption Standard (AES).
- 2022/... – Criptografia Post-Quantum
- ...



Modelo (Confidencialidade)



Alguma terminologia

- **texto-limpo (plaintext)**: conteúdo da mensagem;
- **cifra**: operação que codifica o texto-simples numa mensagem obscurecida — **criptograma (cyphertext)**;
- **decifrar**: operação inversa da cifra
- **sistema criptográfico**: especificação dos algoritmos (possivelmente probabilísticos) que constituem a técnica criptográfica (e.g., geração de chaves; cifrar; decifrar).
- **ataque**: comprometimento dos objetivos de uma técnica criptográfica (e.g., obter a mensagem transmitida sem a chave correspondente).
- **adversário** (intruso; inimigo; ...): personificação do “ambiente hostil”.

Segurança da uma Cifra

- Historicamente, a segurança era essencialmente atribuída ao sigilo associado à técnica subjacente
 - ...o que teve consequências desastrosas!
- Em grande medida, essa tendência manteve-se até meados do século XX. No entanto, já no século XIX, *Auguste Kerckhoff* enunciou o seguinte princípio:

A segurança de uma cifra deve ser avaliada com o pressuposto de que todos os detalhes de sua construção são de conhecimento público

- Corolário: a segurança só pode ser derivada de um parâmetro que é explicitamente mantido secreto - a **chave**



Adversário



- Personalização do **ambiente hostil**
- ...também conhecido pelo nome de **(E)ve; (M)allory**; ...
sucesso do adversário ⇒ **Ataque** a uma técnica criptográfica
- **Segurança Criptográfica** pode ser definida como a impossibilidade de ataques:

Diz-se que uma técnica criptográfica é **segura** se nenhum adversário conseguir atacá-la.

- Na prática, é importante descrever as capacidades do adversário:
 - O controle que ele exerce sobre o canal de comunicação (e.g. read only, read+write):
 - **Passivo** - o adversário só consegue escutar o canal de comunicação;
 - **Activo** - além disso, o adversário tem a capacidade de manipular os dados que circulam no canal de comunicação (modificar / bloquear / inserir ou repetir mensagens).
 - Poder computacional:
 - **Computacionalmente ilimitado** — adversário é capaz de executar qualquer algoritmo instantaneamente (e sem limitação de memória);
 - **Computacionalmente limitado** — adversário só pode executar algoritmos até uma determinada classe de complexidade.
 - Informações adicionais (por exemplo, transcrições de comunicações anteriores);
 - etc.
- Dependendo do tipo de adversário considerado, diferentes noções de segurança (criptográfica) podem ser alcançadas:
 - **Segurança Absoluta (information-theoretical)** — relativa a adversários computacionalmente ilimitados;
 - **Segurança Computacional** — relativa a adversários computacionalmente limitados.

Cifras clássicas

Cifra de César

- Conhecida por ter sido usada por *Júlio César* durante a campanha da Gália
- A cifra consiste numa rotação das letras do alfabeto.

Texto limpo:	A	B	C	D	E	F	...	T	U	V	X	W	Y	Z
Criptograma ($K=6$):	G	H	I	J	K	L	...	Z	A	B	C	D	E	F

- Para decifrar, a rotação é feita na direção oposta.
- O número total de chaves possíveis é 26 (uma das quais é fraca).
- Exemplo: Cifra da mensagem CartagoEstaNoPapo , com chave $K=6$, resulta em IGXZGMUKYZGTUVGVU.

...atacar a cifra de César

- E.g. para atacar o texto cifrado: FXLNTQCLOPNPDL C
- Espaço-chave pequeno sugere a **estratégia #1**:

- decifrar com todas as chaves possíveis...
- ...e identificar o(s) resultado(s) que “fazem sentido”

CRIPTOGRAMA:	FXLNTQCLOPNPDL C
+1	: GYMOURDMPQOQEMD
+2	: HZNPVSENQRPRFNE
...	:
[MSG] +15	: UMACIFRADECESAR (K=26-15=11)
...	:
+24	: DVJLROAJMNLNBJA
+25	: EWKMSPBKONOMOCKB

- **Estratégia #2**: análise de frequência permite ataques mais eficientes...

alta frequência de 'L' sugere que ele corresponda a 'A' (ou seja, $K = 'L' - 'A' = 11$)

Ataque de força bruta

- O ataque da estratégia #1 é conhecido como **Ataque de força bruta**
*o adversário procura todo o **espaço de chaves** na esperança de encontrar a chave certa*
- Assume que:
 - Há redundância suficiente na mensagem original,
 - ou um par de texto-limpo/criptograma é conhecido.
- Frequentemente visto como um ataque que sempre pode ser aplicado a um criptossistema...
- ...mas sua viabilidade depende do tamanho do espaço de chaves.
- **Conclusão**: os tamanhos das chaves são um critério necessário (embora não suficiente) para a segurança das cifras.

Grandes números!

...qual será um tamanho da chave razoável?

- Se considerarmos que as teclas são bitstrings arbitrárias, o tamanho do espaço de chaves aumenta exponencialmente com o tamanho da chave

- Exemplo:

Key Size	Time (1μsec/test)	Time (1μsec/10 ⁶ tests)
32 bit	35.8 min.	2.15 msec.
40 bit	6.4 days	550 msec
56 bit	1140 years	10 hours
64 bit	500000 years	107 days
128 bit	$5 * 10^{24}$ years	$5 * 10^{18}$ years

- Baseline: 2^{112} fornece um nível aceitável de segurança!

Cifra de Substituição Monoalfabética

- Em vez do deslocamento usado na Cifra de César, a cifra executa uma permutação arbitrária do alfabeto (para decifrar, usa-se a permutação inversa);
- E.g.:

A	B	C		W	Y	Z
R	X	K		B	I	F

- Espaço de chaves muito maior ($26! \approx 17.5 * 10^{24}$)

...Devemos confiar na segurança desta cifra?

...um ataque

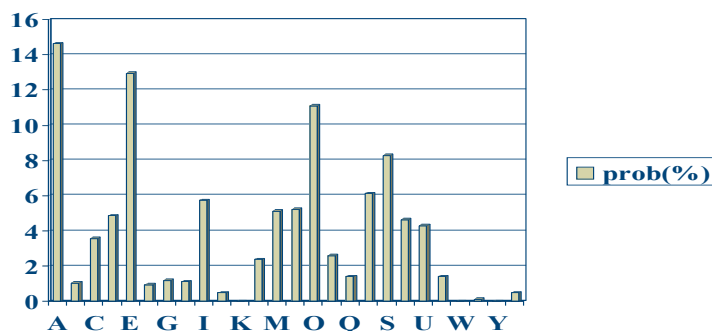
- Considere o seguinte texto cifrado:

FPGFBNEVPKFBDMSBEMDMGUCDKDGUGDMUSPMMDBEFLEFEQDCPPGIDEXDCBKPMDHKMPFQBBUGPSUGHKEGPF
QBMPXPKSESEBSURBHKBBHMEQBFUFSDSBGHKPPFCECPHDKQPDHDKQPFDBADVEDFDCDDCEZPKLDZEDGMPPM
NDKPMGDVPEMPDNUPFQDVMPCPZGEFUQBMCPUGMEOPFSEBHPFBMBFBDUNPCDPXSEQDSDBCBBUQBFBCEPMU
KNEKDUGUDMHPKMBFDNPFMCBPKENPGBIMSUKDSBGJUPGPFQKPQEVPSBSEOEIUIOBMPGKPMQDUKDFQMPX
SPFQKESBMPMPMQKDZEDGUGDHPKNUFQDQPKKEVPOBJUPPJUPVDEMUSPCPKKPMHBFCEOAPMJUPFDBMDIED
PPOPMBOADKDGHDKDBHKDQBSBGEFJUEPQDSDB

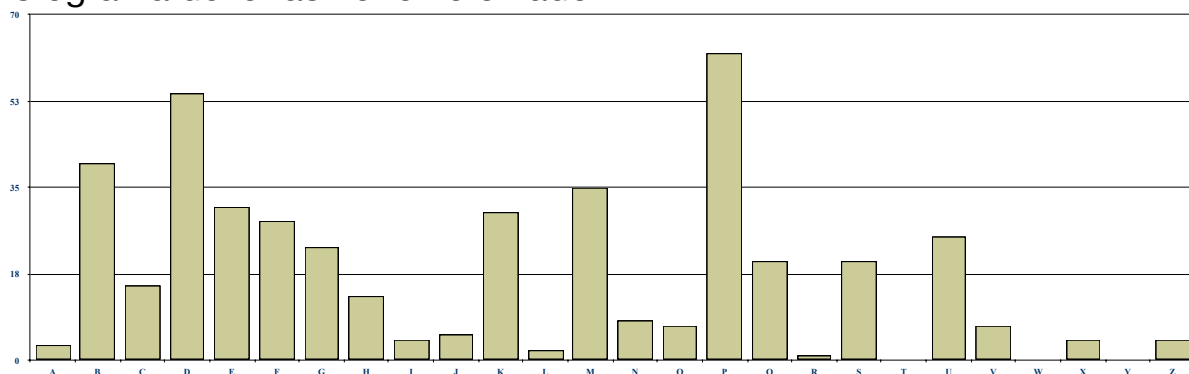
- ...que se sabe ser um texto em português.

Como devemos proceder?

- Frequência de letras do português:



- Histograma de letras no texto cifrado:



- Sugere combinar “A”; “E”; “O” com “P”; “B”; “D”...
- Pares ou triplos de letras também podem ser considerados, como “os”; “es”; “que”; “nao”; ...
- Várias ocorrências dos pares “PM”, “PF”, “MP” e “JUP” sugerem a seguinte decifragem parcial: ({...; **F:M**; ...})

ME-MO-O-E-MOAS-O-SAS-U-A-A-U-ASU-ESSAO-M--M--A-EE--A--A-O-ESA--ESEM-OU-E-U--
---EM-OSE-E-----O-U-O--O-OS--OMUM-A-O---EEM---E-A--EA-A--EMAO-A--AMA-AA---E-
-A--A-SEES-A-ESA-A-E-SEA-UEM-A-ASE-E---MU-OS-EU-S--EM--O-EMOSOMOAU-E-AE----
A-AO-OOU-OMO-ESU----A-U-AS-E-SOMA-EMS-EO---E-O-S-U-A-O-QUE-EM--E---E-OM----
A-U-OSE--ES-AU-AM-ESE--EM----OSESSES--A--A-U-A-E--UM-A-E----E-OQUEEQUE-A-SU-
E-E--ES-OM----ESQUEMAOSA--AEE-ESO--A-A--A-AO--A-O-O--MQU-E-A-AO

- ...o que, numa análise mais detalhada, não parece fazer muito sentido.... :-(

- Precisamos voltar atrás em algumas de nossas hipóteses...
- Reatribuindo a associação de F para N...
- ...temos algo muito mais promissor:

NE-NO-O-ERNOAS-O-SAS-U-ARA-U-ASU-ESSAO-N--N-TA-EE--A--A-ORESARESENTOU-E-U--R--
ENTOSE-ER-----O-U-O-RO-OS-TONUN-A-O--REEN---E-ARTEA-ARTENAO-A--ANA-AA---ER-A--A-
SEES-ARESARESA-A-E-SEA-UENTA-ASE-E---NUTOS-EU-S--EN--O-ENOSONOAU-E-AE---TA-AO-
OOUTONO-ESUR--RA-U-AS-ERSONA-ENS-EOR--E-O-S-URA-O-QUE-ENTRET--E-ON----A-U-OSE-
RESTAURANTESE--ENTR--OSESSESTRA--A-U-A-ER-UNTATERR--E-OQUEEQUE-A-SU-E-ERRES-ON---
-ESQUENAOSA--AEE-ESO--ARA--ARAO-RATO-O--NQU-ETA-AO

- O que leva rapidamente a todo o texto-limpo

NEMNOGOVERNOASCOISASMUDARAMUMASUCESAOINFINITADEEMBAIXADORESAPRESENTOUMECUMPRIMEN
TOSEXERCICIOCUJOPROPOSITONUNCACOMPREENDIDEPARTEAPARTENAOHAVIANADAADIZERFAZIAMSEES
GARESAMAVEISEAGUENTAVASEDEZMINUTOSDEUMSILENCIOPENOSONOAUGEDAEXCITACAODOOOUTONODESU
RGIRAMUMASPERSONAGENSDEORIGEMOBSCURACOMQUEMENTRETIVECONCILIABULOSEMRESTAURANTESEX
CENTRICOSSESSESTRAZIAMUMAPERGUNTATERRIVELoqueEQUEVAISUCEDERRESPONDILHESQUENAOSABIA
EELESOLHARAMPARAOPRATOCOMINQUIETACAO

Cifra de Vigenère

(Substituição polialfabética)



- Atribuído a *Blaise Vigenère* (século XVI). Conhecido como "*le chiffre indéchiffrable*".
- Intercala várias cifras de César.
- Quebrado no século 19 por *Charles Babbage* e *Friedrich Kasiski*.
- Descrição:
 - Chave é uma palavra — cada letra corresponde a uma única chave cifra César (A=0; B=1; ...);
 - Cifrar: aplicar a cifra de César com cada carácter da chave em sequência, recomeçando quando não houver mais caracteres-chave disponíveis.
- Observações:
 - A mesma letra nem sempre é cifrada da mesma maneira (adiciona alguns obstáculos à análise de frequência)
 - ... mas se o texto simples for muito maior do que a chave, padrões no texto simples serão refletidos no texto cifrado.
 - As técnicas de criptoanálise que foram desenvolvidas já exibem algum grau de sofisticação.

Cifra de Transposição

- A criptografia permuta as posições dos caracteres (em vez de alterá-los...)
- E.g.
 - Considere uma permutação [2; 1; 3] (key)
 - para cifrar "AindaOutraCifra", escreva o texto limpo numa matriz...
- ...e leia as colunas (na ordem [1; 2; 3])
- Texto cifrado resultante: "IATCRADUAFNORIA"

2	1	3
A	I	N
D	A	O
U	T	R
A	C	I
F	R	A

- Observação: a análise de frequência agora é inútil!

Composição da Cifra

- Tendo visto diferentes cifras simples...
- ...que isoladamente oferecem um nível de segurança muito limitado...
- será sensato combinar várias destas cifras para obter uma cifra segura?
- **Depende!**
 - Pode acontecer que não acrescente nenhum valor (por exemplo, combinando duas cifras de substituição)
 - No entanto, alguns padrões de combinação podem ser vantajosos (por exemplo, intercalando substituições com permutações — aka *SP-networks*).

One-Time Pad (Vernam Cipher - 1917)



- Generaliza a cifra de Vigenère com:
 1. O tamanho da chave é, pelo menos, o tamanho do texto simples;
 2. A chave é totalmente aleatória;
 3. A chave é usada apenas numa única operação de cifra.
- Normalmente descrito como operando em um alfabeto binário: operação de (de)cifrar é o exclusivo-ou (xor) com o bit de chave correspondente.
$$C_i = T_i \oplus K_i \qquad M_i = C_i \oplus K_i$$
- Demonstrada incondicionalmente segura por *Claude Shannon* — “conhecer o texto cifrado não reduz a incerteza sobre o texto simples”.
- A geração e distribuição de chaves tornam essa cifra inviável em cenários realistas.

Conclusão

- Na criptoanálise, todas as informações disponíveis podem ser usadas, incluindo:
 - Informações parciais sobre a mensagem transmitida;
 - Uso da cifra anterior (por exemplo, mensagens cifradas com a mesma chave);
 - Possíveis fraquezas no uso da cifra (por exemplo, deficiências na escolha da chaves, etc.).
- Embora existam técnicas incondicionalmente seguras, elas têm geralmente requisitos que as tornam impraticáveis.
- A maioria (todas?) das técnicas criptográficas usadas hoje é baseada na segurança computacional, limitando-se por isso as capacidades computacionais do adversário.
- O tamanho da chave de uma cifra deve ser determinado com base no nível desejado de segurança (e.g. 2^{112}), levando em conta a quantidade de tamanho da chave que é “consumida” por técnicas conhecidas de criptoanálise.

Criptografia & Segurança

Propriedades de Segurança

- Concentramos-nos até agora nas cifras — uma primitiva criptográfica cujo objetivo é garantir a confidencialidade.
- Mas a criptografia é usada hoje para fornecer garantias para uma ampla gama de propriedades de segurança:
 - **Confidencialidade** — content of the message is only known to the legitimate parties;
 - **Integridade** — o destinatário "rejeitaria" mensagens que foram adulteradas;
 - **Autenticidade** — garante a "origem" da mensagem para o destinatário;
 - **Não repúdio** — quem originou a mensagem não pode negá-la;
 - **Anonimato** — ausência de informação disponível sobre a “origem” da mensagem;
 - **Identificação** — Estabelecer a “identidade” de uma parte;
 - ...

Serviços e Protocolos Criptográficos

- Normalmente, estamos interessados numa combinação dessas propriedades (por exemplo, um canal seguro entre duas partes visa garantir confidencialidade+autenticidade+integridade).
- Por outro lado, algumas dessas propriedades não resultam da aplicação directa de uma única técnica criptográfica, mas antes de uma combinação de técnicas.
- Levando ao que é conhecido como **protocolos criptográficos** — a especificação da troca de mensagens entre diferentes partes, e que dependem da criptografia para atingir um fim desejado.
- A segurança desses protocolos depende não apenas da segurança das técnicas criptográficas subjacentes, mas também de interações subtis entre elas.

Criptografia & Segurança

A segurança de um sistema que usa criptografia não se resume à segurança das técnicas criptográficas subjacentes.

- Podemos distinguir (pelo menos) os seguintes níveis ao estabelecer a segurança de um sistema que usa criptografia:
 - Esquema criptográfico;
 - Protocolo;
 - Implementação (codificação);
 - Uso.

Uma brecha de segurança em qualquer um desses níveis compromete a segurança de todo o sistema!

Criptografia Aplicada

Criptografia Simétrica

Criptografia Assimétrica

Aplicações

Conteúdo

- Cifras
 - Cifras Sequenciais (stream ciphers)
 - Cifras de Blocos (block ciphers)
- Funções de Sentido único (one-way functions)
 - Funções de Hash criptográficas
 - Códigos de Autenticação (MAC)
 - Funções de Derivação de Chaves (Key-Derivation Functions — KDF)
- Gestão de Chaves (key-management)

Cifras Sequenciais

Recorde o OTP (one-time pad)

- Sabido ser incondicionalmente seguro



- Aspectos relevantes:
 1. As chaves não podem ser reutilizadas
 2. Chave verdadeiramente aleatória com tamanho maior do que o texto limpo
 3. Não promove difusão — informação sobre a estrutura da mensagem pode ser usada para a manipular (troca de bits).
- Dificuldades associadas à geração e distribuição de chaves tornam o OTP inviável em aplicações do mundo real.

Segurança do OTP

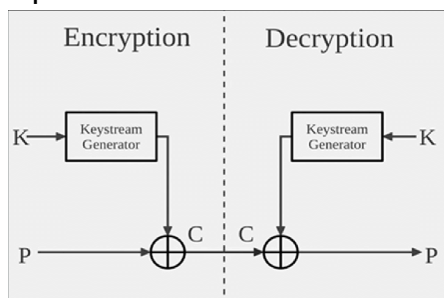
- *Claude Shannon* estabeleceu a segurança absoluta do OTP no artigo “Communication Theory of Secrecy Systems” (1949)
- Segurança é expressa como: “o conhecimento do criptograma não reduz a incerteza relativamente ao texto limpo” — $H(M | C) = H(M)$.
- Propriedade designada por “Segurança Semântica” (semantic security)

Seja $\{0,1\}^n$ o espaço de mensagens, criptogramas, e chaves. Então, para qualquer mensagem m , com $c = k \oplus m$, e quando $k \in K$ é selecionada de acordo com uma distribuição uniforme sobre $\{0,1\}^n$, temos:

$$\Pr[M = m | C = c] = \Pr[M = m].$$

Cifras Sequênciais

- **Ideia base:** aproximar o OTP recorrendo a um gerador pseudo-aleatório responsável por gerar uma sequência de chave de comprimento arbitrário a partir de uma chave curta de comprimento fixo.

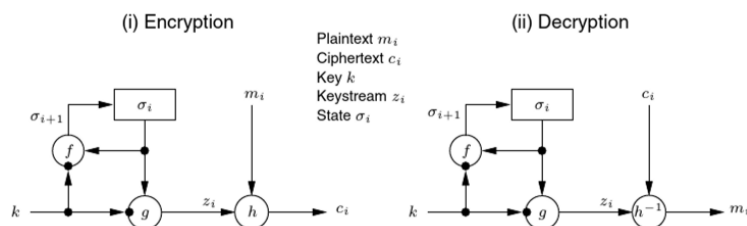


- A geração da sequência de chave deve ser reproduzível (determinística) — uma máquina de estados finitos.
- Portanto, a sequência é necessariamente cíclica — **período** é o comprimento da sequência antes de se repetir.

CrITÉRIOS para o desenho de Cifras Sequênciais

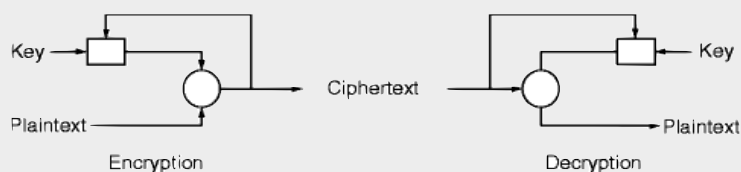
- O período deve ser o mais longo possível (sempre mais longo do que a mensagem a cifrar).
- Sequência de chave deve ser:
 - **pseudo-aleatória:** propriedades estatísticas da sequência devem torna-la indistinguível de uma sequência verdadeiramente aleatória;
 - **Imprevisível:** Deve ser impossível prever o “próximo bit”, depois de observar um determinado prefixo da sequência
- Outras características
 - Sincronismo:
 - **Síncrona** — sequência de chave é independente da mensagem;
 - **Auto-sincronizável** — capaz de recuperar o sincronismo quando bits do criptograma são perdidos.
 - Propagação de erros: o impacto de erros de transmissão nas mensagens cifradas
 - ...

Cifras Sequenciais Síncronas



- *Keystream* é independente da mensagem;
- Perda ou inserção de um bit no criptograma resulta na perda de sincronismo.
- Erros de transmissão (bit swapping) afetam apenas o bit respectivo da mensagem original.
- A chave pode afectar:
 - A função de transição de estado f — **Output-Feedback Mode**.
 - A função de saída g — **Counter Mode**.
 - Ou ambas...

Cifras Sequenciais Auto-Sincronizáveis



- O próximo bit é determinado pelos últimos n bits do criptograma (e chave);
- Um IV (initialisation vector) é usado para iniciar o processo;
- Em caso de erro (*bit-swap* ou inserção/remoção de bit), o sincronismo é recuperado quando tiverem sido consumidos o n bits do *buffer*..
- Possível problema: possibilita ataques por repetição.

Reutilização de chaves e NONCEs

- As cifras sequenciais herdam um problema crítico do OTP: **reutilização de chaves**
 - A cifra de mensagens diferentes com a mesma chave permite obter informação da sequência de chave derivada.
- Esse problema geralmente é superado usando **NONCEs** — abreviatura de **Number used only ONCE**.
 - Um número que nunca deve ser repetido;
 - mas que não precisa ser mantido em segredo (ou seja, pode ser enviado junto com o criptograma).
- Na prática, o Nonce é tipicamente uma sequência de bytes gerados aleatoriamente usados durante a operação de cifra e enviados junto com o criptograma.

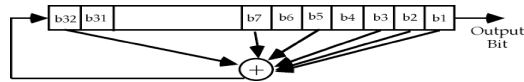
Geradores de Número Pseudo-Aleatórios Criptograficamente Seguros

- Postulados de Aleatoriedade de *Golomb*:
 1. A diferença no número de 1s e 0s deve tender para zero;
 2. O número esperado de sub-sequências de símbolos repetidos (*runs*) com comprimento l é dado por $r(l)=r/2^l$;
 3. A autocorrelação deve ser um valor constante para qualquer desvio distinto de 0 ($\text{mod } p$).
- Segurança Criptográfica:

A sequência gerada deve ser indistinguível de uma sequência aleatória para qualquer adversário PPT (Probabilistic Polynomial Time).

Desenho baseado em LFSRs

- Os Linear Feedback Shift Registers (LFSR) das abordagens mais bem sucedidas na geração de sequências pseudo-aleatórias (em aplicações não criptográficas).
- Um LFSR é um dispositivo “síncrono” (controlado por um “relógio”) onde os bits são deslocados (produzindo um bit de saída) e o bit de entrada é determinado por uma função linear (e.g. $f(x)=x^{32}+x^7+x^5+x^3+x^2+x+1$)



- Características:
 - LFSRs são especialmente apropriados para implementações em hardware.
 - Um LFSR dispõe de período máximo (2^n-1) se e só se o polinômio que lhe está associado for primitivo. Além disso, a sequência resultante satisfaz os critérios de Golomb.
 - ...mas infelizmente não cumprem o requisito de imprevisibilidade (são apenas necessários $2 \cdot n$ bits da sequência para recuperar os parâmetros do LFSR)
- Em aplicações criptográficas, torna-se então necessário “esconder” a forte estrutura matemática que lhes está subjacente.

- Para obscurecer estrutura linear subjacente à utilização de LFSRs, é comum fazer-se uso de uma combinação de LFSRs:

- Utilização de função não linear de combinação:

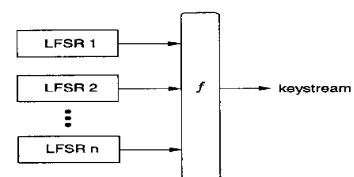


Figure 6.8: A nonlinear combination generator. f is a nonlinear combining function.

- Utilização de LFSRs para regular a cadência de outros LFSRs:

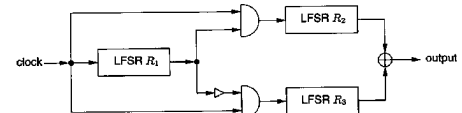


Figure 6.12: The alternating step generator.

- Utilização de LFSRs para filtrar a saída de outros LFSRs:

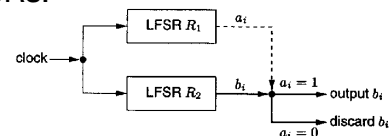


Figure 6.13: The shrinking generator.

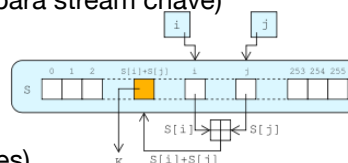
- Multiplexers; Flip-Flops; etc.; etc...

Exemplos de Cifras baseadas em LFSRs

- A5 (A5/2), utilizada no standard GSM
 - Utiliza três LFSRs (de 19, 22 e 23 bits).
 - Já quebrada!!! (2^{40} , 32Gb)
 - Vulgarmente reconhecida como um “bom desenho” mas propositadamente fraco em termos de segurança (registos pequenos e polinómios esparsos)
- E0, utilizado no protocolo Bluetooth (mas já com vulnerabilidades identificadas)
- CSS (Content Scramble System), utilizado na protecção de discos DVD — também quebrado...
- Pode-se afirmar que o desenho de cifras baseadas em LFSRs está, em grande medida, desacreditado (i.e., preterido em relação a outros desenhos mais sistemáticos).

RC4 (arc4)

- RC4 (ArcFour) é uma das primeiras cifras sequenciais síncronas desenhadas para serem executadas em software (1987).
 - Cifra desenvolvida por Ron Rivest (RSA Labs). Originalmente “trade secret”, mas o algoritmo foi divulgado por um post anónimo na [newstnet](#) em 1994 (descoberto por engenharia reversa).
 - Admite chaves de comprimento variável (até 2048 bit), mas não prevê utilização de Nonce
 - Opera em *Output Feedback Mode*:
 - Estado interno (256+2 bytes) inicial obtido a partir da chave;
 - iterada depois função que actualiza estado (e produz 1 bit para stream chave)



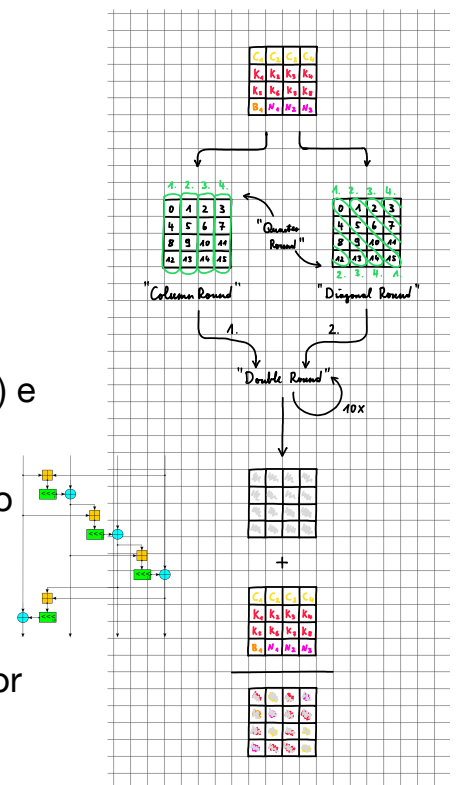
- Considerada insegura (identificadas “demasiadas” vulnerabilidades)
- **É desaconselhado o seu uso!**

Cifras baseadas em PRFs

- A generalidade das cifras sequenciais recentes baseiam-se em *Pseudo-Random Functions* (PRFs)...
- PRFs são funções que dependem de uma chave e que são indistinguíveis de funções aleatórias (para qualquer adversário PPT)
 - Uma chave k determina uma instância particular $F_k: \{0,1\}^n \rightarrow \{0,1\}^n$
 - Sem conhecimento de k , F_k não pode ser distinguido de uma função aleatória.
- A cifra é definida por uma “construção criptográfica standard”, como “*output-feedback mode*” e “*counter mode*”
- Admitem, assim, uma justificação mais rigorosa para o seu desenho (redução à segurança da primitiva PRF).

ChaCha20

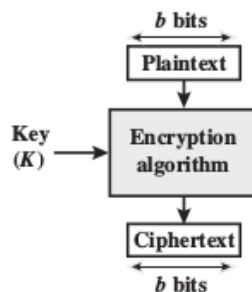
- Cifra sequencial desenhada por *Daniel Bernstein*
- Refinamento da cifra Salsa20 submetida pelo autor à iniciativa eSTREAM (EU)
- Opera sobre estado interno de 512 bit ($4 \times 4 \times 32$ bit). Estado inicial dependente de chave (256 bit), Nonce (64 ou 96 bit) e contador (64 ou 32 bit)
- Opera em 10 rounds, em que cada um destes é composto por 4 execuções paralelas da *quarter-round function*.
- Cada execução processa até 512bit da mensagem
- ...bits seguintes são processados incrementando contador (*counter mode*)



Cifras de Blocos

Cifras de Blocos

- Uma primitiva que se foca no processamento de mensagens de comprimento fixo (um **bloco**)



- A generalização para uma cifra (que lida com mensagens de comprimento arbitrário) é tratada por uma construção padronizada (**modo de operação**).
- Em termos de segurança, espera-se que a primitiva seja indistinguível de uma permutação aleatória (quando k é amostrado aleatoriamente) — (keyed indexed) **pseudo-random permutation (PRP)**.

Cifras de Bloco vs. Sequenciais

- “Unidade de processamento” diferente (bloco vs. bit/byte);
- O uso de cifras de bloco depende sempre de algum modo de operação;
- Cifras sequenciais não promovem a “difusão”;
- A operação inversa de cifra de bloco é mais difícil de obter;
- As cifras sequenciais são tipicamente mais rápidas do que as cifras de bloco;

Modos de Operação Básicos

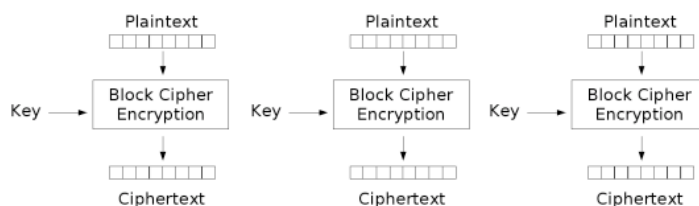
- Uma cifra de bloco é uma primitiva que, por si só, só pode processar bitstrings de comprimento fixo (blocos).
- Um **modo de operação** descreve como a primitiva é usada para processar mensagens de comprimento arbitrário.
- Dependendo das aplicações, podemos escolher entre vários modos padrão:
 - *Electronic Code Book* (**ECB**)
 - *Cipher Block Chaining* (**CBC**)
 - *Counter Mode* (**CTR**)
 - ...
- Cada modo de operação vem equipado com sua própria análise de segurança: suposições; provas de segurança; etc..

Padding

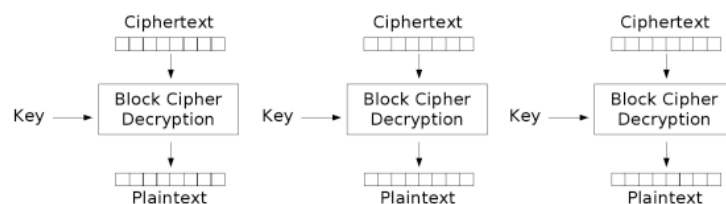
- Certos usos das cifras de bloco (modos de operação) exigem que o tamanho do texto limpo seja um múltiplo do tamanho do bloco.
- Esse requisito é geralmente resolvido adotando um método de **padding**: uma transformação reversível que preenche o último bloco do texto limpo.
 - Várias construções padrão disponíveis:
 - orientado ao bit ou ao byte;
 - determinístico ou aleatório;
 - etc.
 - E.g. PKCS7 padding

PKCS#7 Valid Padding															
A'	B'	C'													
41	42	43	05	05	05	05	05	05	05	05	05	05	05	05	05
A'	B'	C'	D'												
41	42	43	44	04	04	04	04	04	04	04	04	04	04	04	04
A'	B'	C'	D'	E'											
41	42	43	44	45	03	03	03	03	03	03	03	03	03	03	03
A'	B'	C'	D'	E'	F'										
41	42	43	44	45	46	02	02	02	02	02	02	02	02	02	02
A'	B'	C'	D'	E'	F'	G'									
41	42	43	44	45	46	47	01	01	01	01	01	01	01	01	01
A'	B'	C'	D'	E'	F'	G'	H'								
41	42	43	44	45	46	47	48	00	00	00	00	00	00	00	00

Electronic Code Book (ECB)

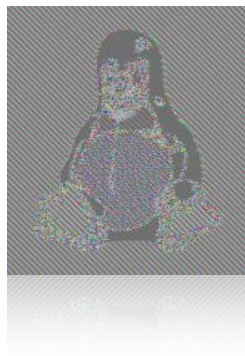


Electronic Codebook (ECB) mode encryption

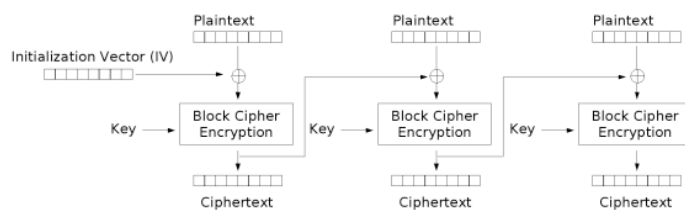


Electronic Codebook (ECB) mode decryption

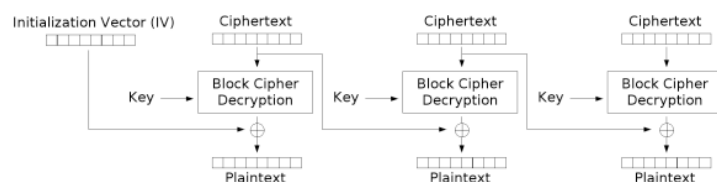
- Requer *padding*.
- A repetição de blocos é detectável — **code book attack**;
- ...possibilitando que padrões do texto simples.
- **Só deve ser usado para cifrar mensagens de um único bloco.**
- Vulnerável a ataques de repetição/substituição.



Cipher Block Chaining (CBC)



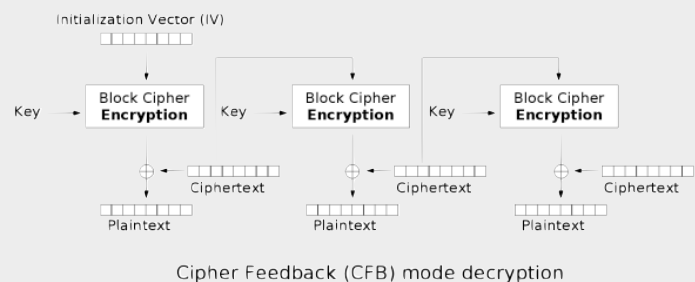
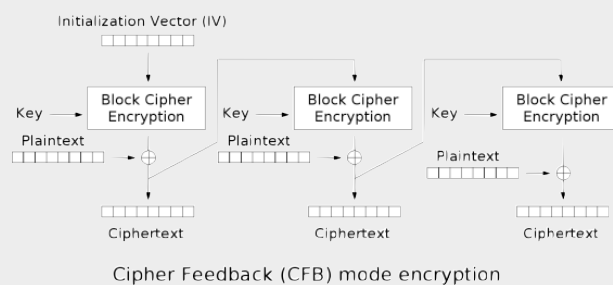
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

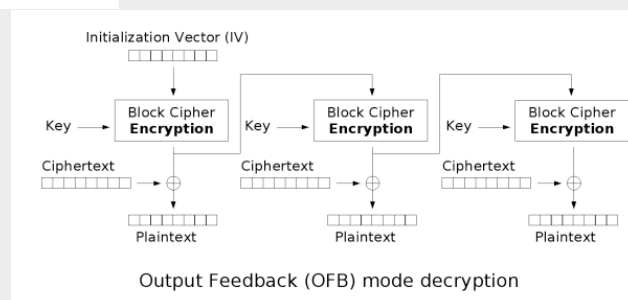
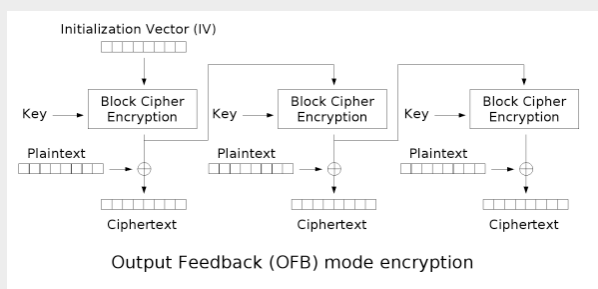
- Cada bloco de texto limpo é mascarado pelo bloco do criptograma anterior
- ...que induz a dependência de todos os blocos de texto limpo anteriores.
- Um **Initialisation Vector (IV)** aleatório deve ser usado para iniciar o processo.
- Como acontece com os *Nonces*, o *IV* não precisa ser secreto.
- A troca de um bit do texto cifrado afeta o bloco correspondente e um único bit do bloco seguinte.
- O último bloco do criptograma pode ser usado como um *Message Authentication Code (CBC-MAC)*.
 - Em tal uso, um IV fixo é adotado;
 - Seguro para mensagens de comprimento fixo (mas passível de ser “corrigido” para suportar mensagens de comprimento arbitrário — e.g. CMAC).

Cipher FeedBack mode (CFB)



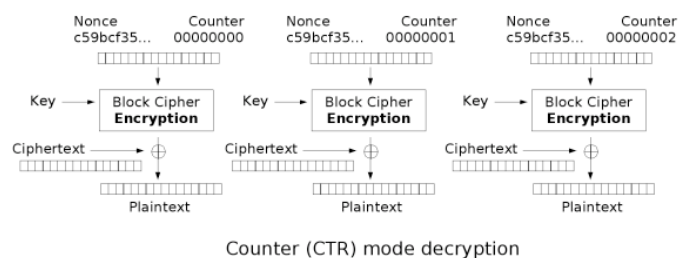
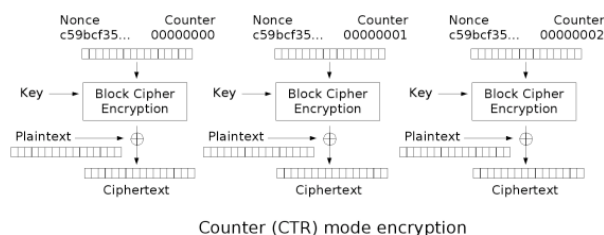
- Implementa uma cifra sequencial auto-sincronizada.
- Não requer *padding*.
- IV deve ser um *Nonce*.
- Observe que a primitiva de criptografia por blocos é sempre usada em modo “encrypt”.
- Keystream depende da chave, IV e de todo o texto limpo anterior.
- O número de bits no *feedback* é variável (CFB_n).
 - Entre 1 e o tamanho do bloco;
 - *Feedback* transfere os n bits mais significativos para os bits menos significativos do bloco (com deslocamento dos restantes bits);
 - O número de bits no feedback afeta o tempo de resincronização.

Output FeedBack mode (OFB)



- Implementa uma cifra sequencial síncrona no modo output-feedback.
 - A cifra por blocos é usada como função de transição de estado (*next-state function*);
 - A função de saída (*output-function*) é a identidade.
- Não necessita de padding.
- IV deve ser um *Nonce*.
- A cifra de bloco primitiva é usada apenas no modo “encrypt”.
- Keystream não depende da mensagem — é obtido ao iterar a cifra de blocos no IV.
 - Pode ser expandida antes do recebimento da mensagem;
 - Mas precisa ser processado sequencialmente.
- A troca de um bit do texto cifrado afeta apenas um único bit.

CounTeR mode (CTR)



- Implementa uma cifra de fluxo síncrono (em *counter mode*).
 - A função do próximo estado é simplesmente o incremento.
 - A cifra em bloco é usada como função de saída.
- Não requer *padding*.
- IV deve ser um Nonce.
- A primitiva é usada apenas no modo “cifrar”.
- A sequência de chave não depende da mensagem.
 - Pode ser expandido antes da recepção da mensagem;
 - e cada bloco pode ser processado de forma independente (e.g., em paralelo).
- A troca de um bit do texto cifrado afeta apenas um único bit na mensagem recuperada.

Cifra Autenticada

- Uma tendência recente tem sido a adoção de modos de operação que combinam garantias de confidencialidade com autenticidade (integridade) dos dados.
- Geralmente suporta também "Dados Associados" — dados que não são cifrados, mas estão anexados à garantia de integridade (e.g., metadados).
- Alguns exemplos:
 - **EAX** (Encrypt-then-Mac-then-Translate)
 - **CCM** (Counter with CBC-MAC)
 - **GCM** (Galois/Counter Mode)
 - **OCB** (Offset CodeBook)

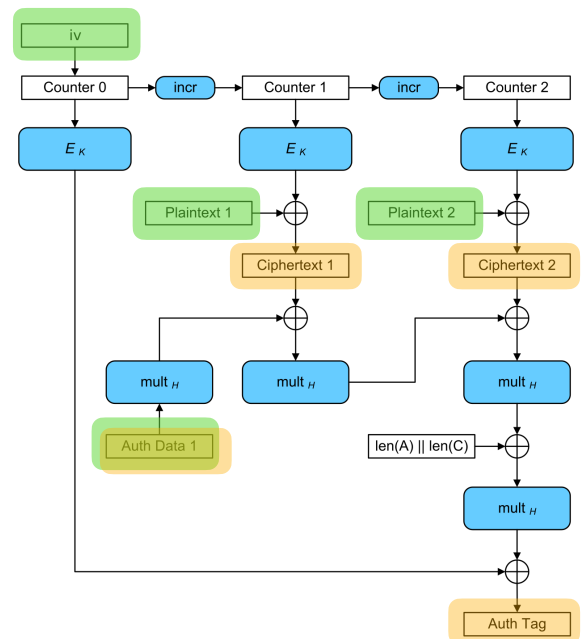
E.g. — Galois/Counter mode

- Estende o modo CTR com um código de autenticação definido a partir da função de hash universal GHASH

$$X_1 \cdot H^m + X_2 \cdot H^{m-1} + \dots + X_m \cdot H$$

em que H é uma chave derivada da cifra de blocos utilizada ($E_k(0^{128})$).

- GHASH avalia um polinómio, cujos coeficientes são os blocos do criptograma, num ponto definido pela chave H (note a utilização do método de *Horner* na avaliação do polinómio)
- Suporta dados auxiliares (AuthData1) para os quais é assegurada a integridade (mas não são cifrados)



Desenho de cifras de bloco

- Duas propriedades desejáveis [C. Shannon 1949]:
 - **Difusão** — a influência das mudanças no texto simples deve ter impacto em todo o texto cifrado;
 - **Confusão** — relação entre o texto cifrado e a chave deve ser o mais complexa possível.
- No mesmo artigo discute-se como podem ser promovidas essas propriedades recorrendo a
 - Substituições (de tamanho razoável):

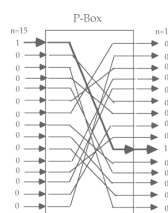


Fig 2.2 - Permutation or Transposition Function

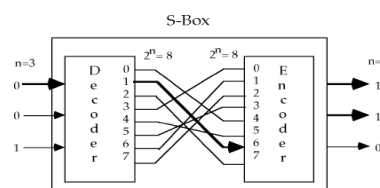


Fig 2.1 Substitution Operation

- Estas construções elementares, quando intercaladas, promovem o que se designa por **efeito avalanche**:

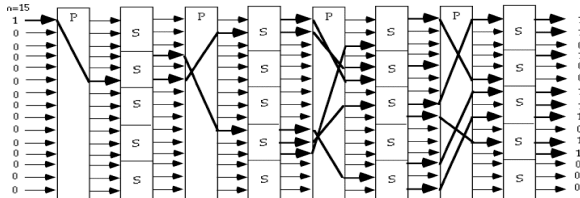
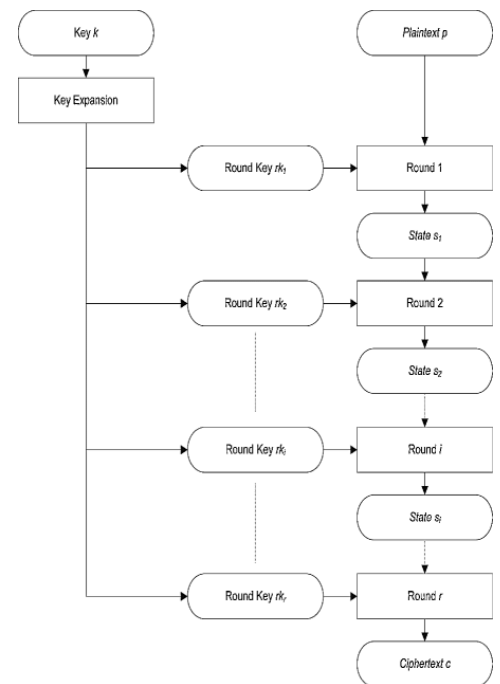


Fig 2.3 - Substitution-Permutation Network, with the Avalanche Characteristic

- A maioria dos desenhos modernos segue o padrão de cifra iterada, em que uma **transformação de ronda** bijetiva (que combina elementos de ambas as construções elementares) é aplicada repetidamente com chaves distintas (derivadas da chave de cifra).



Advanced Encryption Standard (AES)

- Chamada da NIST para padronizar um algoritmo de cifra de blocos — (anúncio: 1997, submissão de propostas: 1998, decisão: 2001).
- Requisitos:
 - Blocos e chaves com, pelo menos, 128 bits;
 - Mais rápido e mais seguro do que o TripleDES (standard anterior);
 - Especificação detalhada e racional do desenho;
 - Adequado para implementações de software (arquiteturas de 32 bits).
- Finalists:
 - MARS** — complex, fast, high security margin
 - RC6** — very simple, very fast, small security margin
 - Rijndael** — clean design, fast, good security margin (**WINNER**)
 - Serpent** — slow, clean, very high security margin
 - Twofish** — complex, very fast, high security margin

AES (Rijndael)

- Tamanho do bloco de 128 bits (16 bytes), organizado como uma matriz de 4x4 bytes;
- Cifra de bloco iterada. Número de rodadas: 10, 12 ou 14 (dependendo do tamanho da chave);
- Suporta chaves de 128, 192 e 256 bits.
- Detalhes da especificação e justificação matemática rigorosa;
- Fases do processamento de cada ronda:

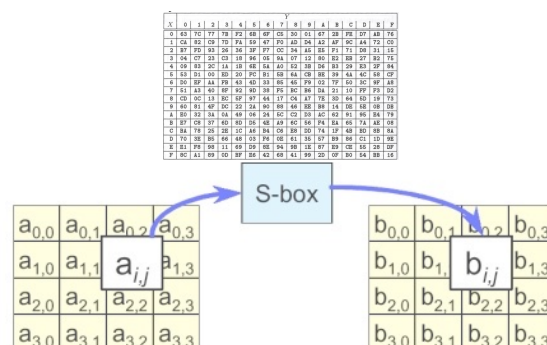
- SubBytes
- ShiftRows
- MixColumns
- AddKey

(com variantes invertidas correspondentes e ajustes subtis para facilitar a implementação)

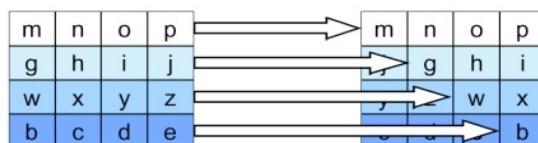
- **Acredita-se ser resistente a todos os ataques conhecidos!**

AES round

- ByteSub
 - A single S-box
 - Highly non-linear

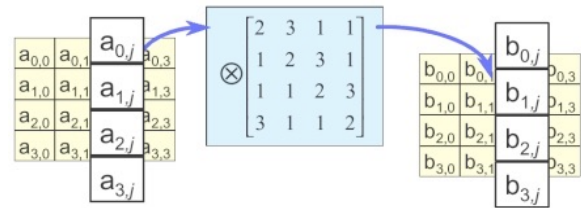


- ShiftRow
 - Rotate rows



- MixColumn

- Interaction with ShiftRow promotes high diffusion in multiple rounds



- KeyAddition

$$\begin{array}{|c|c|c|c|} \hline a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ \hline a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ \hline a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ \hline a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \\ \hline \end{array} + \begin{array}{|c|c|c|c|} \hline k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ \hline k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ \hline k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ \hline k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ \hline b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ \hline b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ \hline b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \\ \hline \end{array}$$

Funções de Sentido Único

Funções de Sentido Único (One-Way Functions)

- A segurança das técnicas criptográficas traduz-se tipicamente no estabelecimento de que uma determinada função é de **sentido único**.

Uma função $f:A \rightarrow B$ diz-se de sentido único quando, dado algum elemento $x:A$, é fácil calcular $f(x):B$, mas, para qualquer $y:B$ na imagem de f , é muito difícil encontrar qualquer $x:A$ tal que $y=f(x)$ (pseudo-inversa de y).

- As noções de "fácil" e "difícil" devem ser entendidas no contexto da teoria da complexidade.)
- De um ponto de vista teórico, a existência de funções OW é geralmente identificada como a suposição que torna a criptografia simétrica viável.

Funções de Hash criptográficas

- Um exemplo paradigmático de funções de sentido único são as funções hash criptográficas.
- Uma **função hash criptográfica** é uma função de sentido único que mapeia strings de bits de comprimento arbitrário para um contra-domínio de comprimento fixo.

$$H : \{0,1\}^* \rightarrow \{0,1\}^n$$

- Olhando para as cardinalidades do domínio/codomínio, temos claramente que todas as funções de hash não são injetivas — **colisões existem sempre!** (Ou seja, pontos m_1, m_2 do domínio mapeados num mesmo valor ($H(m_1)=H(m_2)$)).
 - ...mas, sendo funções OW, encontrar colisões deve ser muito difícil.
- O uso de funções de hash é prevalente na criptografia.

Propriedades

- Os requisitos das funções de hash são geralmente expressos pela seguinte hierarquia de propriedades:
 1. **(First) pre-image resistant:** dado um valor hash h , deve ser inviável obter uma mensagem m tal que $H(m)=h$.
 2. **Second pre-image resistant:** dado uma mensagem m_1 , deve ser inviável obter uma mensagem m_2 distinta de m_1 tal que $H(m_2)=H(m_1)$.
 3. **Collision resistant:** é inviável encontrar mensagens distintas m_1 e m_2 tal que $H(m_1)=H(m_2)$.
- O primeiro simplesmente nos diz que H é uma função OW.
- Mas ao desenhar/escolher funções hash, olhamos tipicamente para a propriedade mais forte — resistência a colisões (não se consegue encontrar qualquer colisão).
- No entanto, a segurança de algumas aplicações pode depender de propriedades mais fracas...

Birthday paradox

- Um resultado bem conhecido da teoria das probabilidades diz-nos que precisamos de um contra-domínio de tamanho razoável para se alcançar a resistência à colisão.

Quantas pessoas precisam se considerar para tornar mais provável do que improvável que pelo menos 2 partilhem o mesmo dia de aniversário?
- ...é suficiente considerar um conjunto com cerca de $\sqrt{366} \approx 19$!!!
- Considerando que tamanhos típicos dos contra-domínios das funções de hash variam entre 128 and 512 bits
- ...concluimos que o ataque do dia de aniversário exigiria cerca de 2^{64} and 2^{256} escolhas aleatórias para encontrar uma colisão.

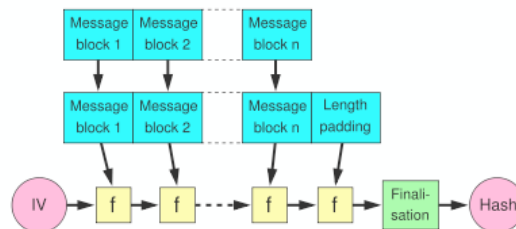
Exemplos

- Funções hash criptográficas mais comuns:
 - **MD5** — tam. contra-domínio: 128 bit. **broken!**
 - **SHA-1** (Standard Hash Algorithm) — tam. contra-domínio: 160 bit. **broken!**
 - **SHA-2** — tam. contra-domínio: 224, 256, 384 e 512 bits. Adote o mesmo desenho de SHA-1, mas com segurança acrescida.
 - **SHA-3** — tam. contra-domínio: 224, 256, 384 e 512 bits. Siga um desenho distinto (sponge construction).
 - **SHAKE-128/256** — eXtendable Output Function (XOF). o mesmo algoritmo de SHA-3.

Desenho de Funções Hash (I)

Construção *Merkle-Damgård*

- Até recentemente, a generalidade das funções hash era baseada na construção *Merkle-Damgård* (e.g. MD5, SHA-1, SHA-2)

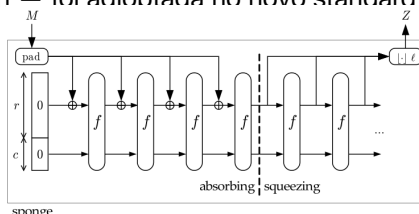


- A função de compressão $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}^n$ que evolui o estado interno (e processa um bloco de mensagens).
- O IV geralmente é específico do algoritmo (constante).
- Pode-se demonstrar que, se f é uma função de compressão livre de colisão, então a propriedade estende-se à função hash resultante.
- É importante que o *padding* inclua informações sobre o comprimento da mensagem.

Desenho de Funções de Hash (II)

Sponge construction

- Avanços na criptoanálise de funções hash baseadas na construção Merkle-Damgård levaram à adoção de construções alternativas.
- Uma proposta recente — Sponge construction — foi adicionada no novo standard de funções de hash (SHA3 — NIST/FIPS 202 - 08/2015)



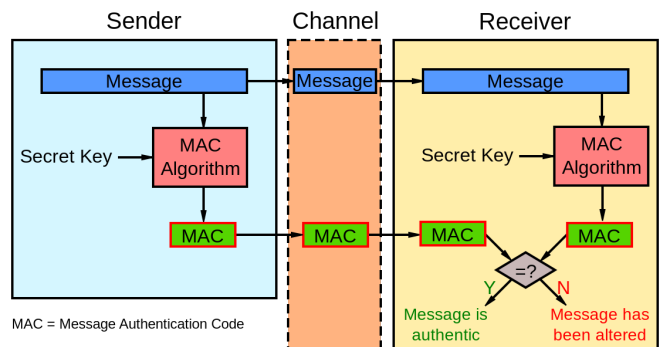
- Processa mensagens e gera resultados de tamanho arbitrário (pode ser usada como uma **XOF** — eXtendable Output-Function)
- Apenas uma fração do estado (*bitrate* r) intervém nas fases em que a entrada é consumida (*absorbed*), e que a saída é gerada (*squeezed out*)
- A parte restante do estado (*capacity* c) é determinante para resistência a colisões
- A construção é segura quando " f " é modelado por uma função aleatória.

Aplicações de funções Hash

- Armazenamento/verificação de passwords;
- Bloco de construção de outras primitivas criptográficas:
 - Message Authentication Codes (MAC);
 - Key-Derivation Functions (KDF);
 - Secure Pseudo-Random Number Generators;
 - Commitment-schemes;
 - Block ciphers;
 - ...

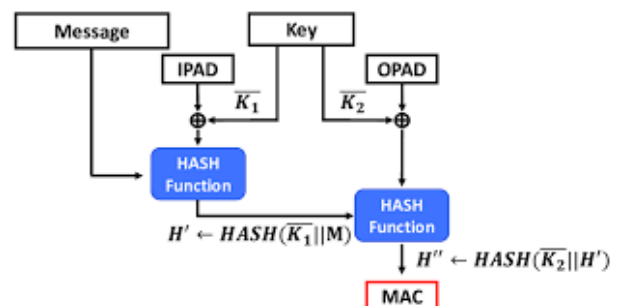
Message Authentication Codes (MAC)

- As funções hash, por si sa, não garantem a integridade dos dados — um usuário mal-intencionado pode recomputar seu valor para os dados modificados.
- Um **Message Authentication Code (MAC)** deve ser usado — este depende de uma chave secreta que é conhecida apenas pelas partes legítimas (remetente e destinatário).
 - o remetente calcula um **authentication tag** (a partir da mensagem e da chave);
 - o receptor recalcula a *tag* e compara-a com a recebida.
- Informalmente, pode ser considerado uma Keyed-Hash (função Hash com uma chave secreta).
- Exemplos: Poly1305; CMAC.



HMAC

- Um MAC pode ser facilmente construído a partir de qualquer função Hash.
- Uma construção padrão é a Hash-based Message Authentication Code (HMAC).
 - executa duas passagens na função hash, para precaver possíveis fraquezas das funções usadas (ataques de extensão).
- Exemplos: HMAC-SHA256; etc.
- Observação: o design SHA-3 não é vulnerável a ataques de extensão. Por isso, a construção aninhada HMAC é redundante para a família SHA-3 — é suficiente usar $H(K||M)$.



Key-Derivation Functions (KDF)

- Key-Derivation Functions (KDF) permite a derivação de chaves criptográficas a partir de outro material (secreto).
- KDFs são usados em uma variedade de situações:
 - Derivar chaves de segredos fracos (passwords/passphrases);
 - Derivar chaves para diferentes fins criptográficos de um único “*master secret*” (key diversification);
 - Derivar chaves de comprimento diferente das fornecidas;
- Às vezes, também aceita entradas não secretas (other-info), permitindo vincular o segredo ao processo de geração.

Password-Based KDF

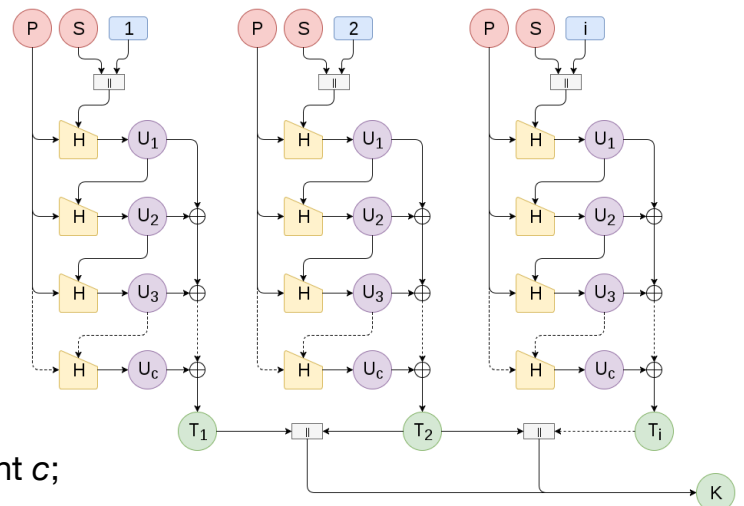
- Passwords/Passphrases (senhas) **nunca devem** ser usadas diretamente como chaves criptográficas
- ...mas podemos derivar chaves criptográficas dessas senhas (através de uma KDF).
- O principal problema é que uma KDF não ultrapassa a sua fraquezas intrínseca:
 - low entropy;
 - vulnerabilidade a *dictionary attacks*.
- Apesar de todas as suas desvantagens, as senhas continuam sendo um recurso prevalente na segurança da informação (autenticação de utilizadores; proteção da *keystores*; etc.)

- Recomenda-se que um KDF, a ser usado com segredos fracos, implemente contramedidas que minimizem os problemas mencionados acima:

- utilizar um **salt** — um número aleatório que randomiza a saída da função OW-function. Protege contra ataques baseados na pré-computação (e.g. rainbow tables);
- deve ser **resource intensive** (CPU time and/or memory) para dificultar os ataques ao dicionário.

- Exemplo: PBKDF2

- Paramétrico num dado MAC H ;
- utiliza salt S and um iteration count c ;
- possibilita a derivação de segredos de tamanho arbitrário.



Protecção de Passwords

- Certain applications, and specifically *password protection*, recommend stricter counter-measures to dictionary attacks.
 - account for the possibility of attackers build dedicated password-cracking machines;
 - go beyond computational intensive: *memory-hard* designs.
- Some algorithms:
 - **bcrypt** (1999) — support adjustable cost;
 - **SCrypt** (2008) — combines PBKDF2 with a special construction (ROMix) making it resistant to hardware brute-force attacks;
 - **Argon2** (2015) — winner of the Password Hashing Competition.

Security balance

- ...but in the end, security will always depend on the entropy of the passphrase!

Estimated cost of hardware to crack a password in 1 year.

KDF	6 letters	8 letters	8 chars	10 chars	40-char text
DES CRYPT	< \$1	< \$1	< \$1	< \$1	< \$1
MD5	< \$1	< \$1	< \$1	\$1.1k	\$1
MD5 CRYPT	< \$1	< \$1	\$130	\$1.1M	\$1.4k
PBKDF2 (100 ms)	< \$1	< \$1	\$18k	\$160M	\$200k
bcrypt (95 ms)	< \$1	\$4	\$130k	\$1.2B	\$1.5M
scrypt (64 ms)	< \$1	\$150	\$4.8M	\$43B	\$52M
PBKDF2 (5.0 s)	< \$1	\$29	\$920k	\$8.3B	\$10M
bcrypt (3.0 s)	< \$1	\$130	\$4.3M	\$39B	\$47M
scrypt (3.8 s)	\$900	\$610k	\$19B	\$175T	\$210B

Gestão de Chaves

Chaves Criptográficas

- Um factor crítico na segurança das técnicas criptográficas é a qualidade das chaves:

Utilizar sempre **Cryptographic Secure Random-Number Generators!**

- Sua forma (e em particular, seu tamanho) depende da técnica específica. Para criptografia simétrica, as chaves geralmente são strings de bits aleatórias (tamanhos que variam entre 128 e 256 bits).
- Deve ser tomado um cuidado extremo ao manipular chaves criptográficas (storage; backup; disposal; etc.)

Manipulação de Chaves

- Como regra:
 - o *tempo de vida* de uma chave (**cryptoperiod**) deve ser tão curto como possível,
 - e inversamente proporcional à sua utilização e criticalidade da informação a proteger..
- Chaves podem ser classificadas como:
 - **Long-term** keys (aka **static** keys);
 - **Short-term** keys (aka **ephemeral** keys).
- Em criptografia simétrica são tipicamente utilizadas *short-term keys*.
- E.g. o estabelecimento de um canal seguro deve basear-se em *chaves de sessão*:

Uma **chave de sessão** é uma chave simétrica de utilização única para proteger a comunicação durante uma interação específica (uma **sessão**).

Distribuição de Chaves

- A pré-distribuição de chaves é o maior desafio no uso de criptografia simétrica.
- Normalmente depende de canais seguros, que são custosos.
- Note a circularidade:

A criptografia pode ser usada para estabelecer um canal seguro entre duas partes, mas depende da existência de um canal seguro para distribuir a chave secreta.

- Se nos lembrarmos da recomendação de usar chaves de sessão, o problema agrava-se ainda mais!
- Iremos ver que a criptografia assimétrica oferece soluções mais satisfatórias, mas é de qualquer forma instrutivo considerar como o uso da criptografia simétrica pode obviar o problema.

Protocolos de acordo de chaves

- Numa comunidade com N utilizadores, o número de chaves que cada membro precisa armazenar para permitir uma comunicação segura entre qualquer par de membros é $(N * (N - 1))/2$.
 - Isso claramente não é viável numa comunidade de tamanho considerável (e.g. internet).
- **Protocolos de Acordo de Chaves** recorrem a uma "rede de confiança" para distribuir as chaves entre as partes interessadas em comunicar.
 - E.g.: uma Trusted Third-Party (TTP) partilha uma chave com cada membro;
 - TTP gera e distribui uma chave de sessão entre qualquer par de agentes a pedido.
- Mas estes protocolos não são adequados para comunidades abertas (como a Internet)...

Princípios para a gestão de chaves

- As chaves não devem ser usadas para fins múltiplos (Em vez disso, KDFs devem ser usadas para derivar chaves distintas);
- Em operações críticas de segurança, o manuseio de chaves (geração; armazenamento; uso; etc.) deve ser limitado a **Hardware Security Modules (HSM)**.
- Por regra, as chaves devem ser destruídas quando comprometidas ou expiradas (mas diferentes cenários de aplicação podem justificar políticas diferentes)
- Se o mecanismo de backup de chaves criptográficas for necessário, deve ser usado um *threshold secret-sharing scheme*.

Cryptographic Key-Management Systems (CKMS)

- Um CKMS consiste em um conjunto de políticas, componentes e dispositivos que são usados para proteger, gerir e distribuir chaves criptográficas.
- Um CKMS cobre todo o ciclo de vida das chaves, i.e. os estados que uma chave passa desde a sua geração até à sua destruição — **key lifecycle**

